

IO网络模型

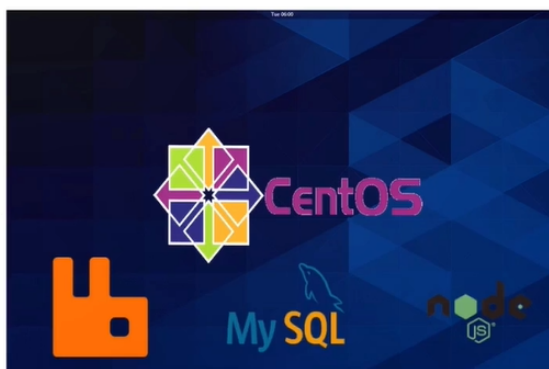
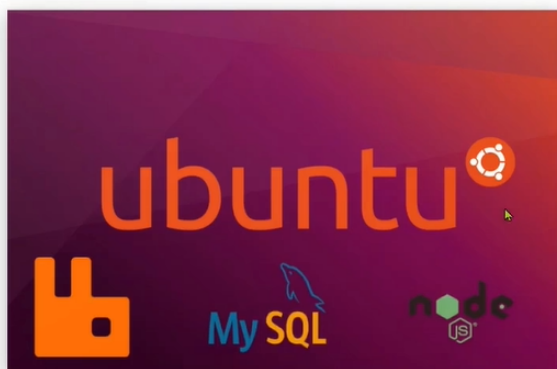
学习IO网络模型，需要了解linux系统五种不同的IO模型：阻塞IO、非阻塞IO、IO多路复用、信号驱动IO、异步IO。

一、用户空间与内核空间

1.1 概念

服务器大多都采用Linux系统，我们以Linux为例来讲解：

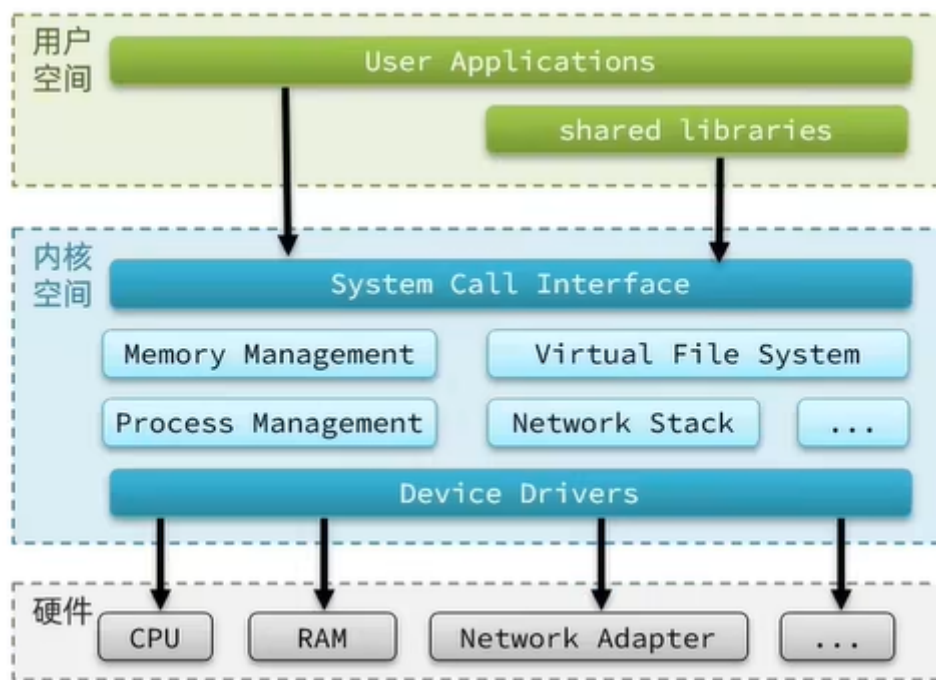
Linux系统有两大发行版本：ubunu和CentOS



发行版可以看做是Linux的基础上包了一层壳，任何Linux发行包，其系统内核都是Linux，我们的应用都需要通过Linux内核与硬件交互。（即最上层的用户应用是无法直接和计算机硬件打交道的，需要通过内核去和与计算机硬件打交道）



计算机硬件主要包含CPU、RAM、网卡（Network Adapter）.....等硬件设备，内核需要通过不同的驱动来操作硬件，在驱动的基础上，可以形成对内存的管理、进程管理、文件管理、网络管理等等，提供系统调用接口来让用户应用进行使用，从而使得用户能够间接对硬件进行访问。



内核本质也是应用，在使用过程中需要CPU、内存等资源，同样用户应用也在使用CPU、内存等资源，如果不加以限制，用户应用可以随意使用，可能会出现冲突甚至内核崩溃，因此用户应用要与内核隔离开来，隔离方式如下所示：

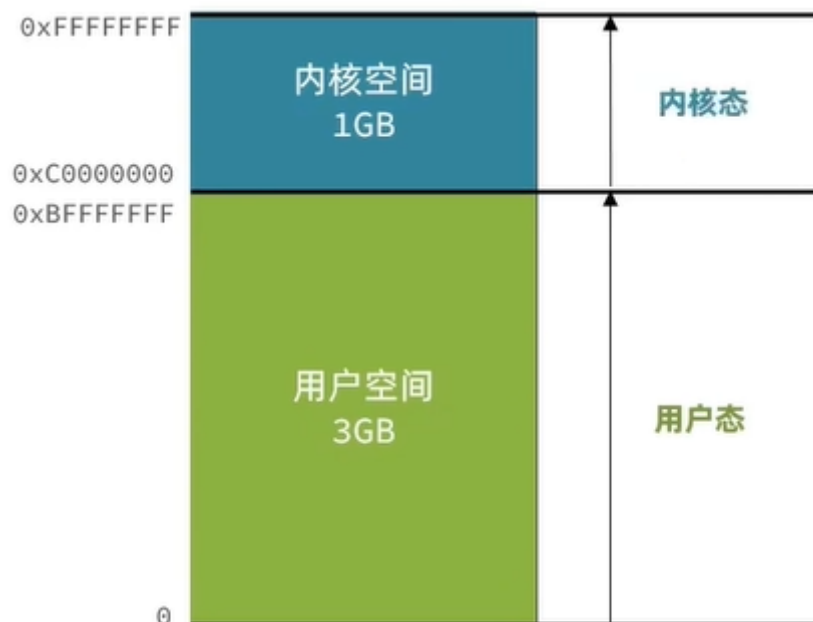
- 进程的寻址空间（即虚拟内存地址）会划分为两部分：内核空间和用户空间

不管是内核应用还是用户应用，都无法直接访问物理内存，而是给他们分配不同的虚拟内存空间，通过**地址映射**（Address Mapping）的机制映射到对应的物理内存空间。每个应用和内核都有自己的虚拟内存空间（虚拟地址是操作系统和硬件通过虚拟内存管理机制提供给应用程序和内核的逻辑地址。它并不直接对应于物理内存的实际位置，而是通过**地址映射机制**（通常是分页或分段机制）映射到实际的**物理地址**，操作系统通过虚拟内存机制使每个进程看到的内存空间是连续的，而且不与其他进程的内存空间干扰。虚拟内存地址对于每个进程来说是唯一的，进程运行时，通过虚拟内存地址来访问内存），这个地址是无符号的整数，最大值取决于CPU的地址总线和寄存器带宽，如32为计算机，最大值就是 2^{32} ，即寻址方位是 $0 \sim 2^{32}$ 。寻址地址会被划分为两部分，高位存放的是内核空间，低位存放的是用户空间：



- 用户空间只能执行受限的命令（Ring3），而且不能直接调用系统资源，必须通过内核提供的接口来访问。
- 内核空间可以执行特权命令（Ring0），调用一切系统资源。

用户应用一般运行在用户空间，内核应用一般运行在内核空间，但是一个进程在执行过程中，可能去执行一些普通命令，也可能是一些特权命令，调用系统资源，因此需要再用户空间和内核空间中进行切换，当一个进程运行在用户空间的时候将其称为用户态，当运行在内核空间的时候，就称其为内核态：

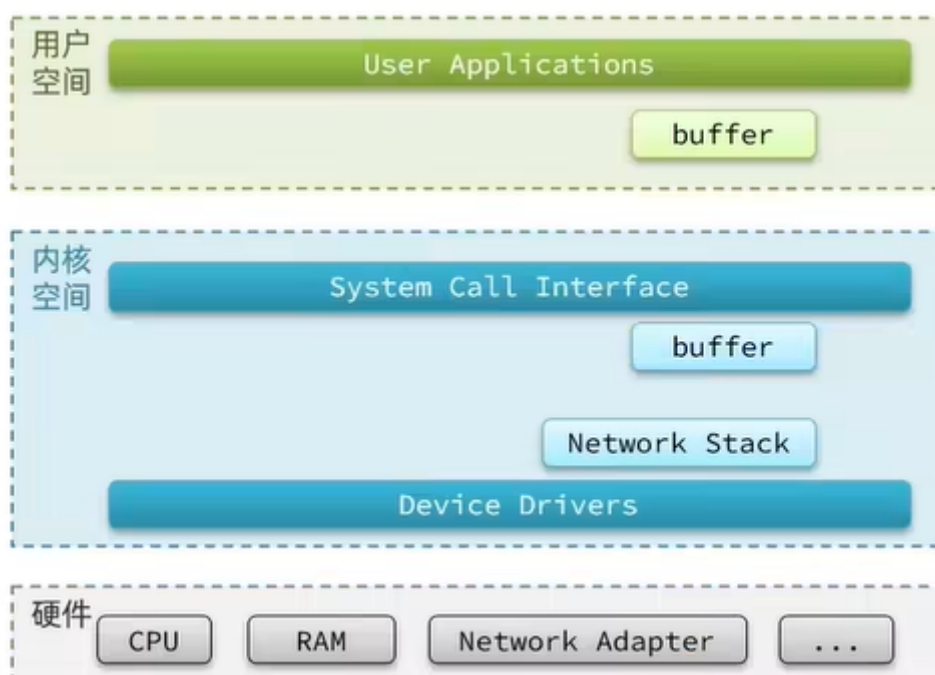


1.2 进程在用户空间和内核空间切换示例

以IO为例，读写会涉及到磁盘、网卡等硬件的访问，即涉及到内核态。

Linux系统为了提供IO效率，会在用户空间和内核空间都加入缓存区，用户空间的缓存区：用于IO流中先写入缓存区，再一次性写出去，读也是如此。

内核空间的缓存区：也是起到一个缓存的作用，如进程去内核中读数据的时候可以缓存在内核区，然后下次读的时候，如果有可以直接读取。

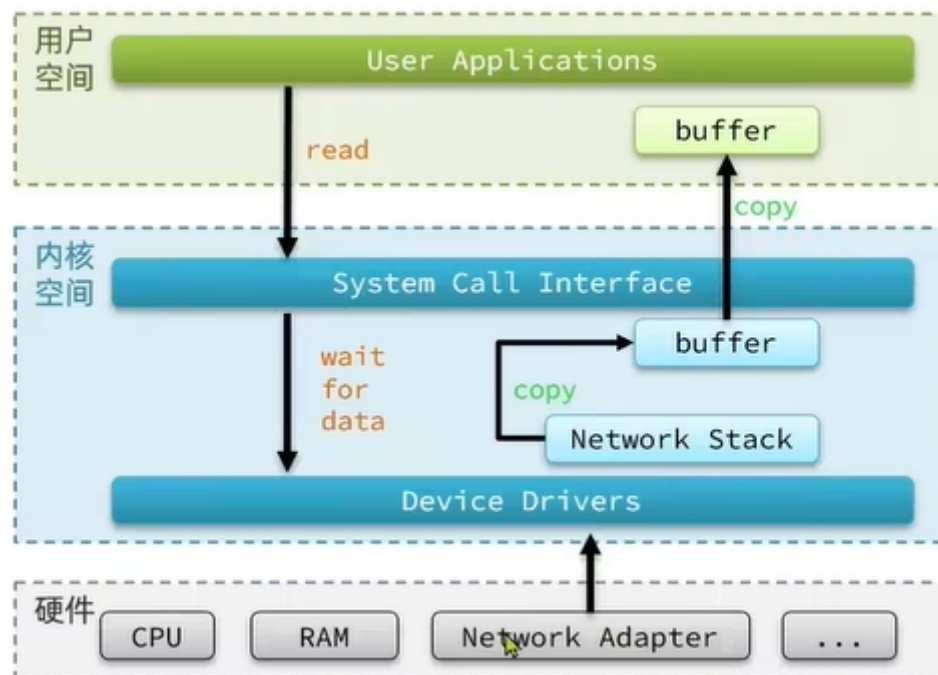


对于一些数据的运算等操作只需要在用户空间进行即可，但是当这些运算完后，需要将数据写出到磁盘，就需要调用内核。其调用过程如下：

- 先将数据写入到用户空间的缓存区
- 将用户空间缓存区中的数据拷贝到内核空间的缓存区
- 切换到内核
- 将内核空间中的数据写入到磁盘

从磁盘/网络中读取也是一样

- 进程在用户态发起读命令
- 切换到内核
- 等待数据（从磁盘或网卡）
- 读取数据到内核缓存区中
- 将缓存区中的数据拷贝回用户空间的缓存区
- 切换用户态
- 对数据执行一些操作



总结：

- 写数据时，要把用户缓冲数据拷贝到内核缓冲区，然后写入设备
- 读数据时，要从设备读取到内核缓冲区，然后拷贝到用户缓存区。

后续IO模型解决问题：

五种linux的IO模型，就是解决上述中数据在内核态和用户态缓冲区之间的拷贝以及数据等待过程中带来的开销。

1.3 关于五种IO模型

在《UNIX网络编程》一书中，总结归纳了5种IO模型：

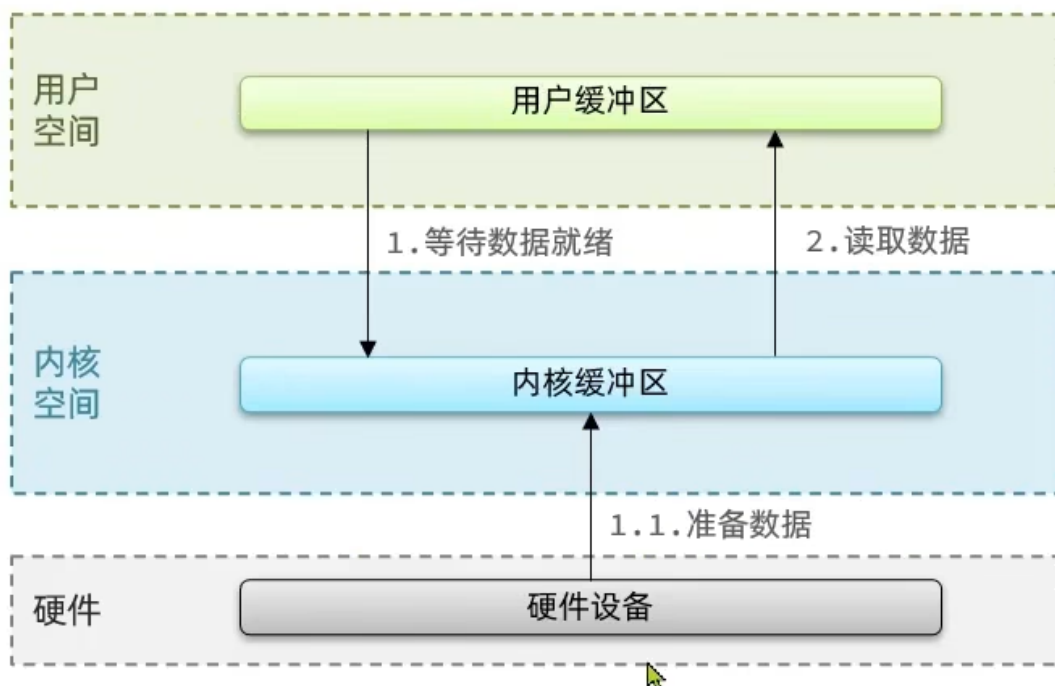
- 阻塞IO (Blocking IO)
- 非阻塞IO (Nonblocking IO)
- IO多路复用 (IO Multiplexing)
- 信号驱动IO (Signal Driven IO)
- 异步IO (Asynchronous IO)

以读取数据为例，分析他们的差异：

整个用户应用读取数据的过程可以概括成两部分：

1.等待数据就绪 2.读取数据（即将内核中的数据拷贝到用户缓存区）

当我们用户应用想要读取硬件设备上的数据时，不管是读取磁盘还是网卡等，都是没有直接的权限去进行操作的，所以必须去调用内核提供的函数，如果内核缓存区的数据还没有准备好，用户应用就需要1.等待数据就绪，只有当磁盘寻址完成，并将磁盘中的数据读取到内核缓存区了，此时等待数据才就绪，之后就可以将内核缓存区的数据2.读取到用户缓冲区中，关于1.等待数据就绪最核心的就是1.1等待硬件的数据到内核缓存区。



对于1.等待数据就绪 和 2.读取数据的处理方式不同，归纳出了不同的IO模型。

二、五种IO模型

2.1、阻塞IO

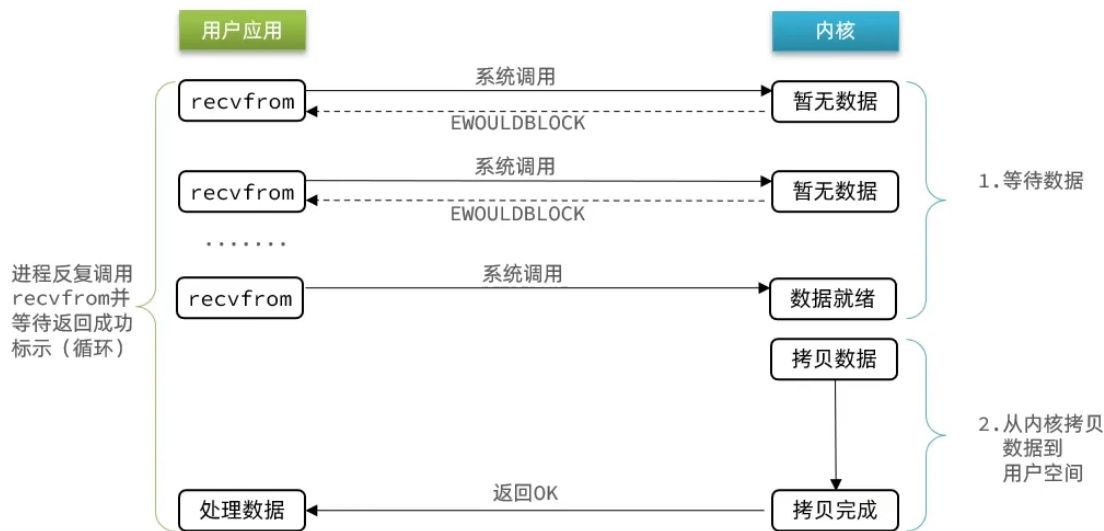
顾名思义，阻塞IO就是两个阶段都必须阻塞等待：



即用户通过recvfrom命令开始等待内核缓存区中的数据就绪，只有等内核数据就绪，并将内核缓冲区的数据拷贝到用户空间，此时用户应用才完成数据的获取。可以看到，阻塞IO模型中，用户进程在两个阶段都是阻塞状态。

2.2、非阻塞IO

顾名思义，非阻塞IO的recvfrom操作会立即返回结果而不是阻塞用户进程



非阻塞IO模型中，用户进程在第一个阶段（即等待数据）过程中是非阻塞的，第二阶段（即读取数据，从内核拷贝数据到用户空间）是阻塞状态。虽然是非阻塞的，但性能并没有得到提高，用户进行一直在调用recvfrom接口，并获取响应结果，这种忙等机制会导致CPU控制，CPU使用率暴增。

但是在某些特殊场景下必须使用非阻塞IO，如后面的IO多路复用，就需要结合非阻塞IO才会有更好的表现。

2.3、IO多路复用

1.引出

无论是阻塞IO还是非阻塞IO，用户应用在一阶段都需要调用recvfrom来获取数据，差别在于无数据时的处理方案：

- 如果调用recvfrom时，恰好没有数据，阻塞IO会使进程阻塞，非阻塞IO使CPU空转，两者都不能充分发挥CPU的作用。
- 如果调用recvfrom时，恰好有数据，则用户进程可以直接进入第二阶段，读取并处理数据

比如服务端处理客户端Socket请求时，在单线程情况下，只能依次处理每一个socket，如果正在处理的socket恰好未就绪(数据不可读或不可写)，线程就会被阻塞，所有其它客户端socket都必须等待，性能自然会很差。

类似于服务于给顾客点餐：

- 顾客思考吃什么（等待数据就绪）
- 顾客想好了，开始点餐（读取数据）

那么提高效率的方法有哪些呢？

- 增加更多服务员（多线程），弊端是开销大，CPU需要做上下文切换
- 不排队，谁想好了吃什么（即数据就绪），服务于就给谁点餐（用户去内核缓存区读取数据）

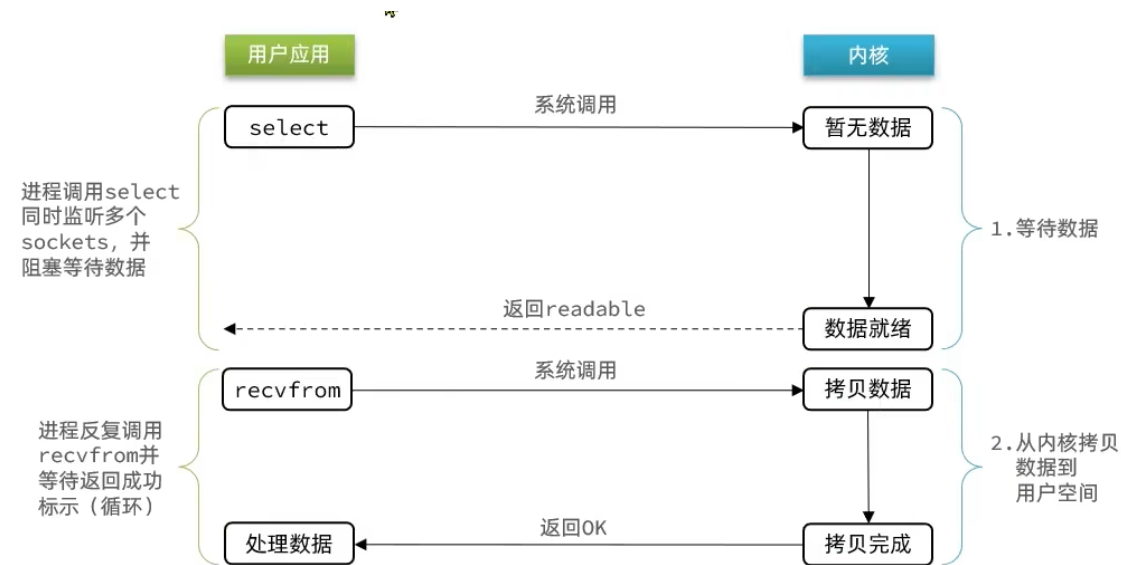
对于方法二，用户进程如何知道内核中数据是否就绪？

文件描述符(File Descriptor):简称FD，是一个从0 开始递增的无符号整数，用来关联Linux中的一个文件。在Linux中，一切皆文件，例如常规文件、视频、硬件设备等，当然也包括网络套接字(Socket)，因此每一个客户端和服务端连接的时候都会有一个对应的文件描述符。

IO多路复用就是利用单个线程来同时监听多个FD，并在某个FD可读、可写时得到通知，从而避免无效的等待，充分利用CPU资源。

2.实现过程

其调用过程如下所示：



其等待数据的过程中，用户应用使用的是Select函数（内部接受多个fd，并传递到内核进行监听，只要有任意一个准备好了就返回readable，但如果所有fd都没有准备，就会进入等待），而不是recvfrom函数（直接去读数据，具体指定到某一个fd，但是去读取的时候不知道对应的fd数据有没有就绪），在select函数返回后，就肯定有fd准备就绪，调用recvfrom，就可以直接从内核缓存区中拷贝数据到用户缓存区，即读取数据（如果有多个fd准备就绪，这里会进行一个循环）

3.IO多路复用的实现方式

- select
- poll
- epoll

select和poll只知道有数据准备好了，epoll在知道数据准备好的同时可以直接定位到对应的人。

4.IO多路复用-select

select是Linux中最早的IO多路复用实现方案

1.select源码

首先我们看一下select源码如下所示

```
// 定义类型别名 __fd_mask, 本质是 long int
typedef long int __fd_mask;

/* fd_set 记录要监听的fd集合, 及其对应状态 */
typedef struct {
    // fds_bits是long类型数组, 长度为 1024/32 = 32
    // 共1024个bit位, 每个bit位代表一个fd, 0代表未就绪, 1代表就绪
    __fd_mask fds_bits[__FD_SETSIZE / __NFBITS];
    // ...
} fd_set;

// select函数, 用于监听多个fd的集合
int select(
    int nfds, // 要监视的fd_set的最大fd + 1
    fd_set *readfds, // 要监听读事件的fd集合
    fd_set *writefds, // 要监听写事件的fd集合
    fd_set *exceptfds, // 要监听异常事件的fd集合
    // 超时时间, null-永不超时; 0-不阻塞等待; 大于0-固定等待时间
    struct timeval *timeout
);
```

select 函数参数说明:

- nfds: 用于记录所监视的fd集合中最大值+1, 即用于在遍历fd集合的时候, 明确到哪一个停止。
- readfds: 需要监听读事件的fd集合 (Linux系统将IO可能发生的事件分为三类, 分别是可读、可写、异常, 因此会有三个fd集合)
- writefds: 需要监听写事件的fd集合

- exceptfds: 需要监听异常事件的fd集合
- timeout: 即调用select等待fd就绪（即1.等待数据）过程中的超时时间，null-永超时；0-不阻塞等待；大于0固定等待时间；

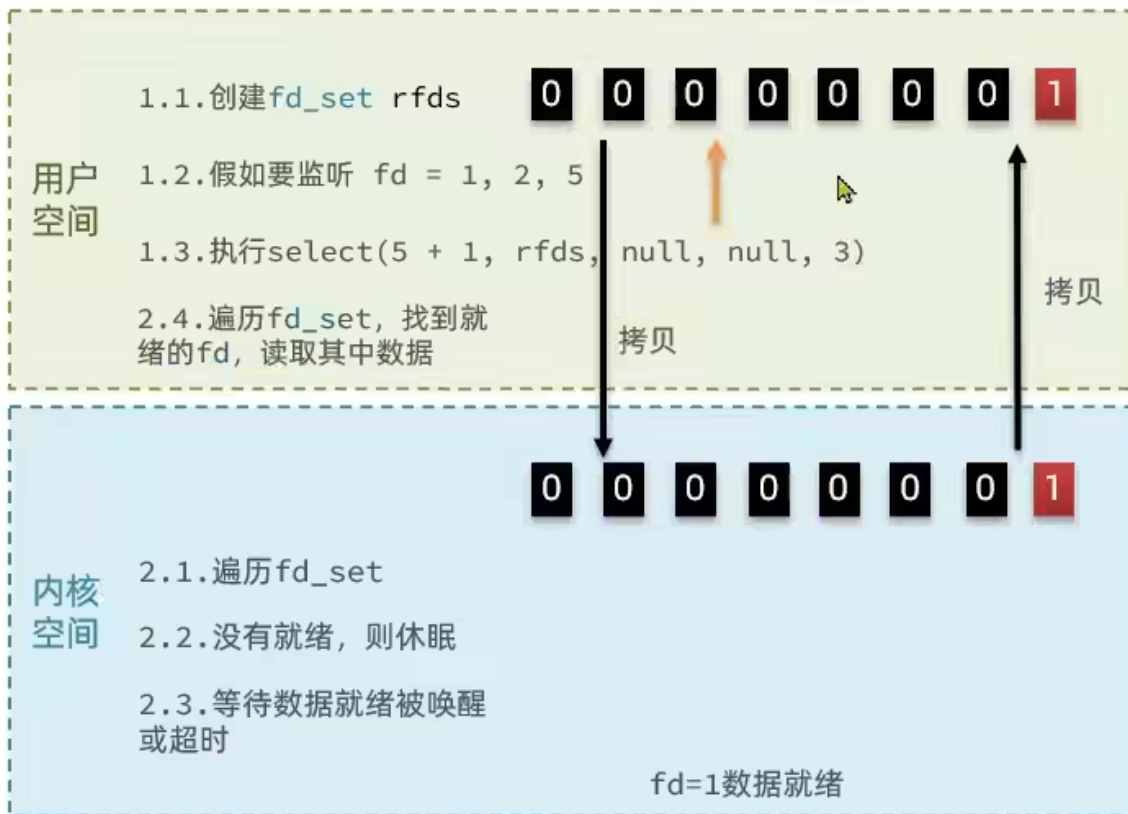
关于其中的结构体fd_set:

其中维护了一个存储的fd数组，即fds_bits，其声明的长度为 $1024 / 32 = 32$ ，其类型为long int,长度为32个比特位，因此这个数组的长度就是1024bit为，每个fd都占一个bit位，因此一共可以存放1024个fd，其中0代表未就绪，1代表就绪。初始化的时候，每个bit位都是0。

2.select执行过程示例:



- 首先用户空间创建fd_set结构体集合对象，并初始化（默认都为0）
- 然后根据需要监听的fd，对fd_set进行赋值。
- 然后调用select方法，并传入对应的入参，在执行select方法那一刻，需要内核去完成fd是否就绪的查看，因此就需要将用户空间中的fd_set集合拷贝到内核空间中（这里即是用户态到内核态的切换，保护数据的拷贝）



- 首先从最低位开始遍历一遍 `fd_set`, 直到遇到最大值即 `nfds`, 查看当前被标记的 `fd` 是否就绪 (如果有就绪就可以直接返回结果给用户应用了)
- 没有就绪, 就休眠, 后台监听这些 `fd`, 等待任一数据就绪后 (示例中是任一数据可读, 因为可写和异常都没有传), 被唤醒或超时
- 如果此时 `fd=1` 的数据就绪了, 就遍历一遍 `fd_set`, 找到所有被标记的 `fd` (即监听的 `fd`), 保留已经就绪的, 未就绪的删除 (改成0)
- 将修改的结果拷贝回用户空间的 `fd_set`, 并将就绪的结果返回给用户应用 (即 `select` 的响应结果)
- 用户空间调用 `select` 只知道就绪的数量, 而不知道哪几个就绪了, 就需要遍历 `fd_set`, 找到就绪的 `fd`, 并读取其中的数据
- 到此, 一个执行过程就完成了, 后面就继续从1.2开始循环执行。

3. 缺点:

- 需要将整个 `fd_set` 从用户空间拷贝到内核空间, `select` 结束后还要再次拷贝回用户空间
- `fd_set` 的设计虽然节省了内存, 但是无法直观地告诉用户应用哪些就绪了, 需要额外地进行开销遍历判断。这也是 `select` 性能比较差的原因。
- `fd_set` 监听数量固定, 并且监听的数量不能超过1024, 监听数量太少

5.10 多路复用-poll

`poll` 模式对 `select` 模式做了简单改进, 该性能提升不明显。

1.poll源码

```
// pollfd 中的事件类型
#define POLLIN      //可读事件
#define POLLOUT     //可写事件
#define POLLERR     //错误事件
#define POLLNVAL    //fd未打开

// pollfd结构
struct pollfd {
    int fd;           /* 要监听的fd */
    short int events; /* 要监听的事件类型：读、写、异常 */
    short int revents; /* 实际发生的事件类型 */
};

// poll函数
int poll(
    struct pollfd *fds, // pollfd数组, 可以自定义大小
    nfds_t nfds, // 数组元素个数
    int timeout // 超时时间
);
```

pollfd结构变量:

- fd: 具体监听的fd
- events: 要监听的事件, select中按照不同的事件, 分为读事件的fd集合、写事件的fd集合和异常事件的fd集合, 在poll模式下, 通过pollfd结构体类型中的events变量去进行区分不同事件, 都放在同一个数组下进行传递。
- revents: 内核在监听过程中, 如果发现事件有就绪, 就会把就绪的事件类型放到revents中, 如果没有事件发送默认给0

(fd、events在调用poll方法的时候需要指定, revents是在内核监听过程中赋值的)

poll函数参数:

- fds: 存放fd和对应的事件类型信息, 可以自定义大小 (其指向 struct pollfd 数组的指针)
- nfds: 数组元素个数
- timeout: 超时时间

2.执行流程

- 创建pollfd数组, 向其中添加fd信息 (需要赋值fd、events变量值), 数组大小自定义。
- 用户应用调用poll函数, 将pollfd结构体数组拷贝到内核空间, 内核会将其转为链表存储, 没有上限大小
- 内核遍历fd, 判断是否就绪
- 数据就绪或超时后, 拷贝pollfd数组到用户空间, 返回就绪fd数量n
- 用户进程判断n是否大于0, 如果大于0就遍历pollfd数组, 找到就绪的fd

3.select比较

- select模式中的fd_set集合大小固定为1024，而pollfd在内核中采用链表，理论上无上限

在内核中，pollfd 使用链表存储，理论上没有文件描述符数量上限。

在用户空间，pollfd 使用数组存储，大小由用户指定，并且没有像select 那样的固定限制，但仍然受限于系统的资源和文件描述符上限。

- 监听FD越多，每次遍历消耗时间也越久，性能反而会下降

6.IO多路复用-epoll

epoll模式是对select和poll模式的改进

1.epoll源码

- eventpoll结构体

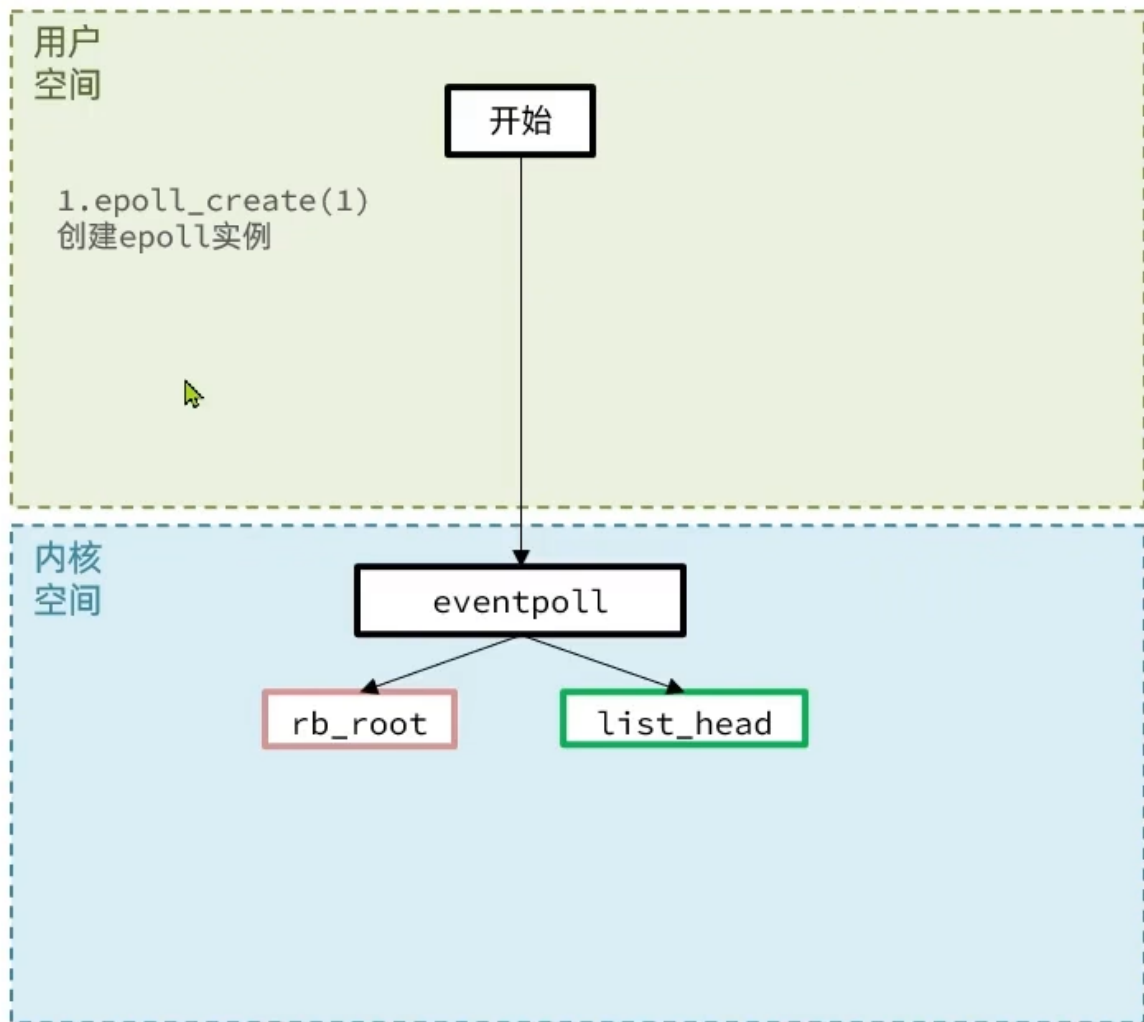
```
struct eventpoll {  
    //...  
    struct rb_root rbr; // 一颗红黑树，记录要监听的FD  
    struct list_head rdlist; // 一个链表，记录就绪的FD  
    //...  
};
```

所有要监听的fd通过红黑树的方式存储在rbr中，就绪的fd存储在链表rdlist中。

- epoll_create函数

```
int epoll_create (int size)
```

会在内核创建eventpoll结构体，返回对应的句柄epfd（看做eventpoll的唯一表示）

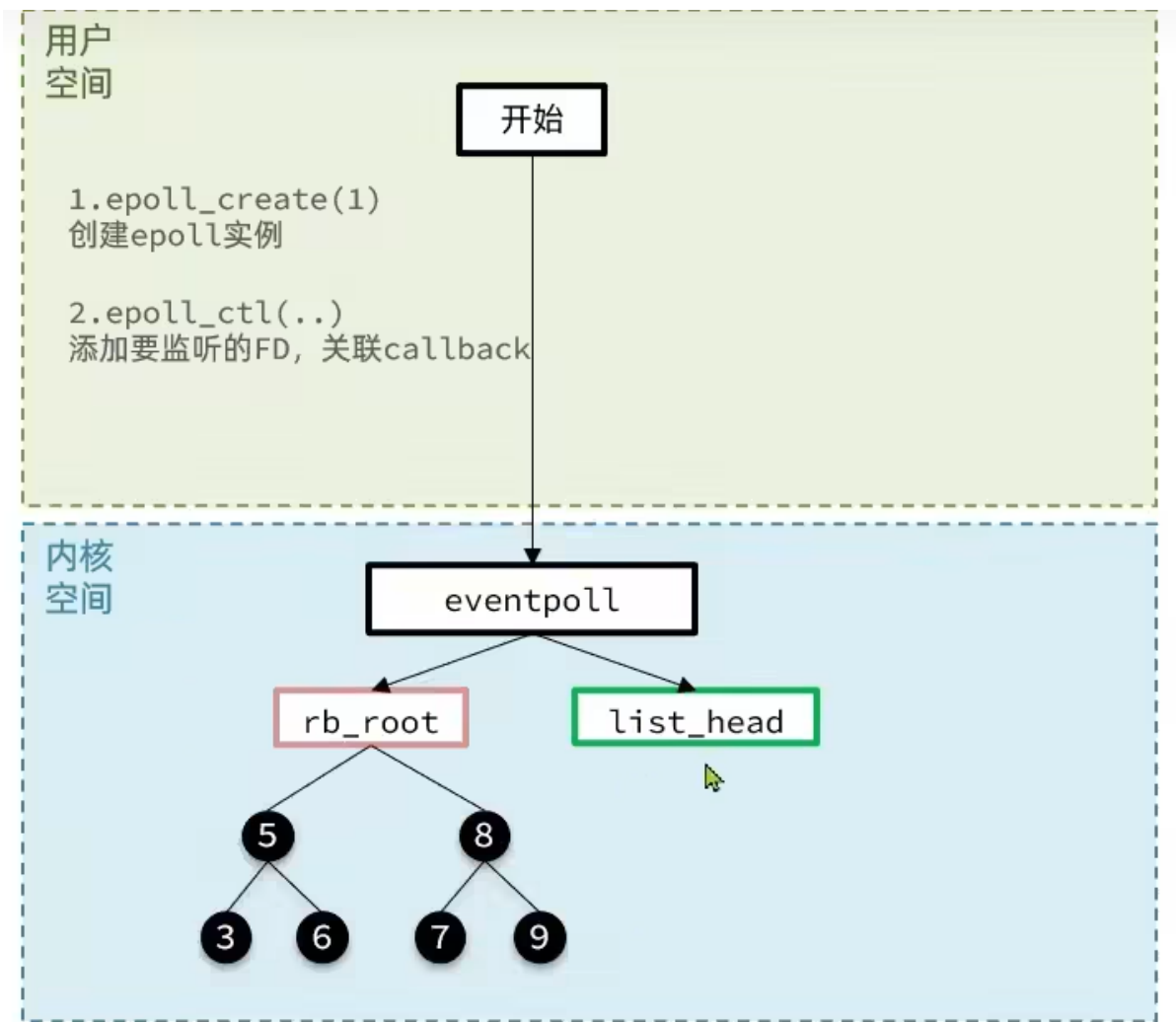


- `epoll_ctl`函数

```
// 2. 将一个FD添加到epoll的红黑树中, 并设置ep_poll_callback
// callback触发时, 就把对应的FD加入到rdlist这个就绪列表中
int epoll_ctl(
    int epfd, // epoll实例的句柄
    int op,   // 要执行的操作, 包括: ADD、MOD、DEL
    int fd,   // 要监听的FD
    struct epoll_event *event // 要监听的事件类型: 读、写、异常等
);
```

就是向eventpoll结构体中维护的rbr中进行fd增删改的操作函数

新增的时候会为fd色后置`ep_poll_callback`回调函数（就相当于监听），即在fd就绪的时候主动触发，其主要就是就爱那个fd加入到rdlist就绪队列中去



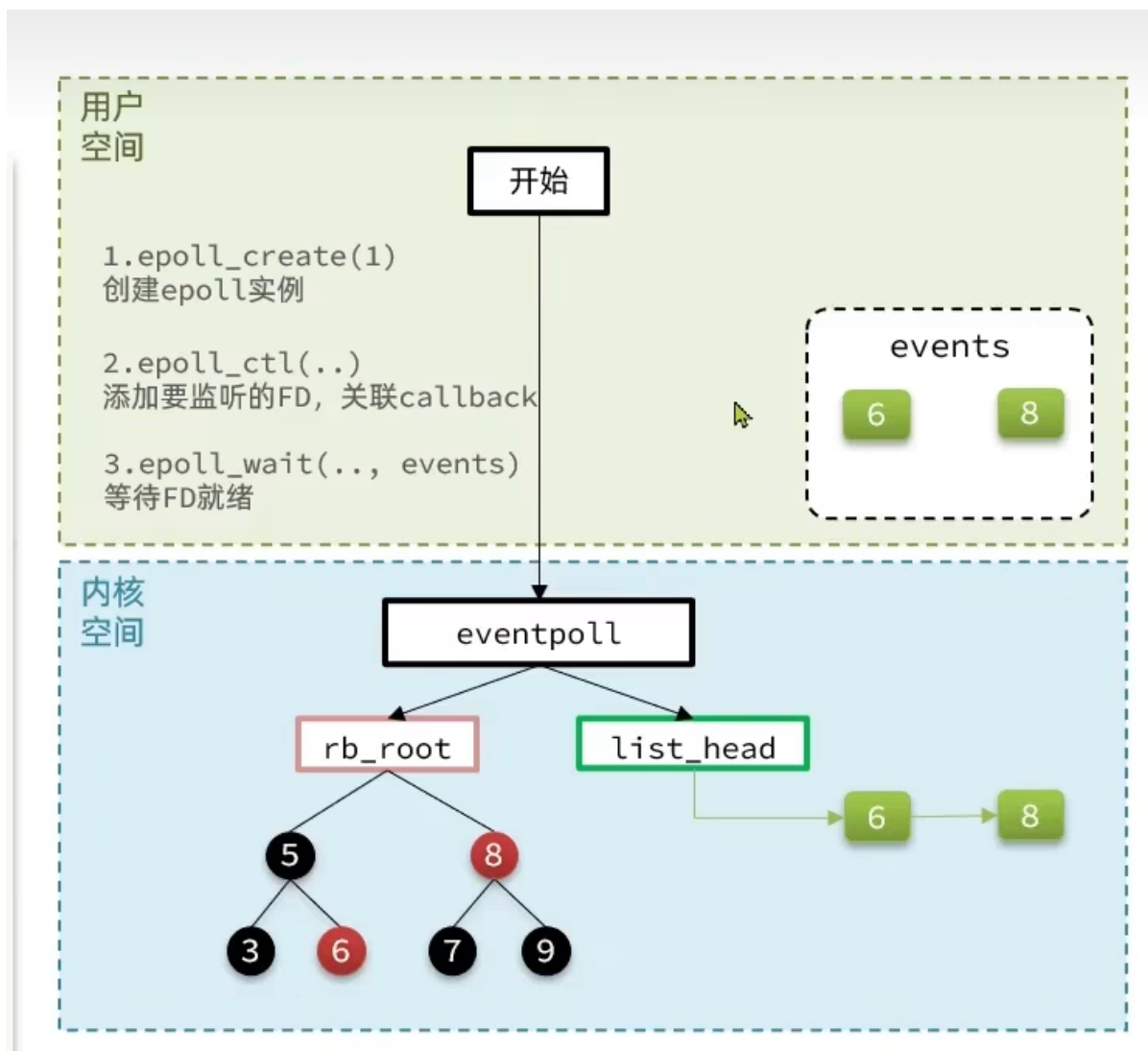
- `epoll_wait`

```
// 3.检查rdlist列表是否为空, 不为空则返回就绪的FD的数量
int epoll_wait(
    int epfd,                // eventpoll实例的句柄
    struct epoll_event *events, // 空event数组, 用于接收就绪的FD
    int maxevents,            // events数组的最大长度
    int timeout               // 超时时间, -1用不超时; 0不阻塞; 大于0为阻塞时间
);
```

参数说明:

- `epfd`: `eventpoll`实例的唯一标识, 创建时候生成
- `events`数组: 存储内核中需要返回的就绪的fd内容
- `maxevents`: 防止内核往`events`数组里面写数据超了, 因为内核是以链表的方式维护的
- `timeout`: 该函数的超时时间 -1不超时 0不阻塞 大于0阻塞时间

该方法返回就绪元素数量, 具体就绪的fd通过`events`数组获取



2. 优点

- 减少拷贝次数和数量
- 不用遍历
- select监听数量，poll遍历需要时间
- 允许监听的竟可能多的fd

3. 关于select模式和poll模式的问题和epoll是如何解决的

select模式存在三个问题：

- 能监听的FD最大不超过1024
- 每次select都需要把所有要监听的FD都拷贝到内核空间
- 每次都要遍历所有FD来判断就绪状态

poll模式存在的问题：

- poll利用链表解决了select中监听FD上限的问题，但依然要遍历所有FD，如果监听较多，性能会下降。

epoll模式如何解决这些问题：

- 基于epoll实例中的红黑树保存要监听的FD，理论上无上限，而且增删改查效率都非常高，性能不会随监听的FD数量增多而下降
- 每个FD只需要执行一次epoll_ctl添加到红黑树，以后每次epoll_wait无需传递任何参数，无需重复拷贝FD到内核空间
- 内核会将就绪的FD直接拷贝到用户空间的指定为止，用户进程无需遍历所有FD就能知道就绪的FD是谁

4.事件通知机制

当FD有数据可读的时候（即list_head中有内容），我们调用epoll_wait就可以返回就绪的FD信息，即得到通知，事件通知的模式有两种：

- LevelTriggered：简称LT。当FD有数据可读时，会重复通知多次（即通知的时候先将对应的FD从list_head中断开，然后如果没处理完，再接回去，下一次调用epoll_wait的时候，又会将list_head中未读取的数据通过events返回给用户空间），直至数据处理完成（Epoll的默认模式）
- EdgeTriggered：检查ET。当FD有数据可读时，只会被通知一次（通知的时候先将对应的FD从list_head中断开，不管有没有处理完都不会再接回去了），不管数据是否处理完。

LT模式示例：

1. 假设一个客户端socket对应的FD已经注册到了epoll实例中
2. 客户端socket发送2kb数据
3. 服务端（用户空间）调用epoll_wait，得到通知说FD就绪
4. 服务端从FD读取了1kb数据
5. 回到步骤3（再次调用epoll_wait，形成循环）

LT模式的缺点：

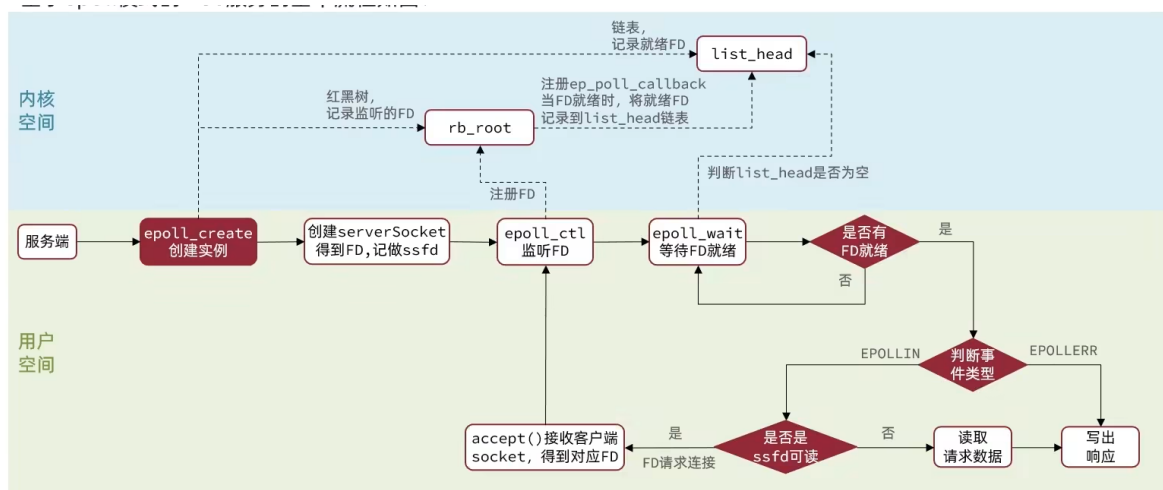
- 因为LT是种重复通知，对于效率和性能会有影响
- 惊群现象，即一个fd就绪，所有监听该fd的进程都会被唤醒，如十个进程都监听了fb_root下的某些fd，当这些进程都调用epoll_wait尝试获取就绪的fd，结果这些fd正好都就绪了，就会去通知这些监听的进程，并且LT模式下，任何一个进程通知完成以后，fd还会存在于list_head中，这也导致了所有监听这些fd的进程都会被唤醒，在实际处理中，可能前几个进程就已经完成了数据的处理，后面没有必要再去唤醒其他的进程了。

ET模式手动实现数据读取方式：

- 在第一次调用epoll_wait读取完数据后（即过程3），还有数据的情况下，通过epoll_ctl函数将未读取数据的fd修改一下，此时eventpoll会去再检查一下对应fd是否有未读取的数据，即可读数据，然后将这些fd再添加到list_head中，这样就完成了未读取数据的手动添加，之后再手动调用epoll_wait又可以获取未读完的fd
- 在一次通知中，循环将fd中的数据读取完成。这里要注意循环处理要使用非阻塞IO的方式（即读fd有数据立马返回，没数据也返回对应的标识），而不能用阻塞IO（因为阻塞IO读fd，如果fd没有数据了会一直在这里）

因此ET模式虽然实现手动读取数据起来可能麻烦一点，但是其避免了LT模式可能出现的惊群现象，其最好结合非阻塞IO读取FD数组，相比LT会复杂一些。

7.10多路复用-web服务中的流程

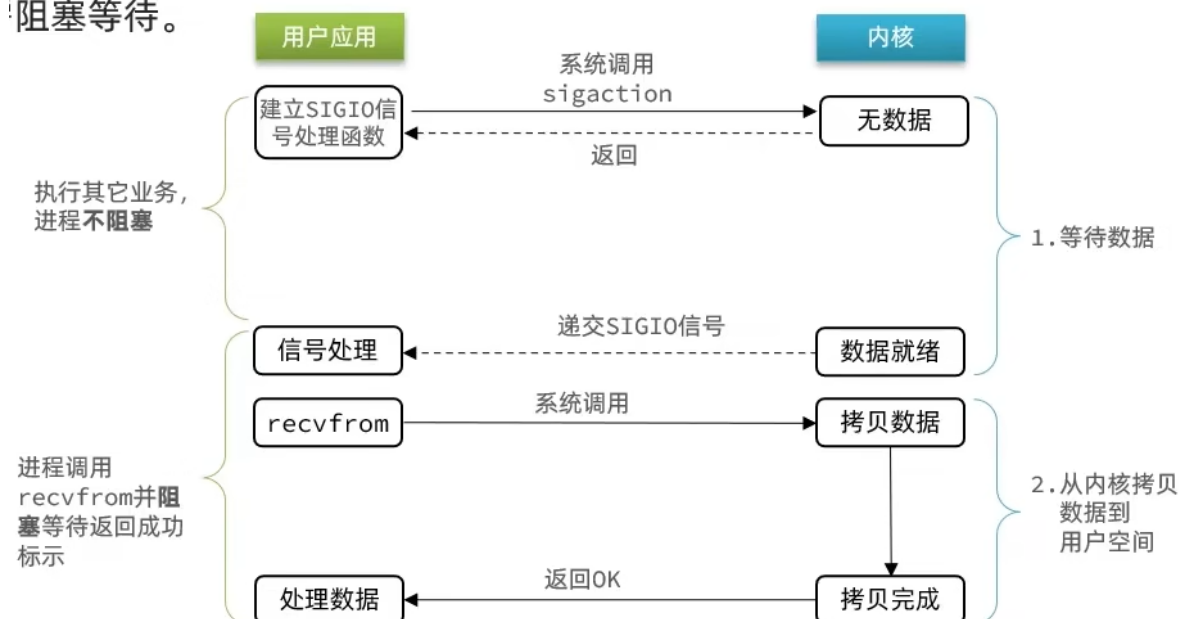


- 首先服务端调用epoll_create创建eventpoll实例 (红黑树rb_root&记录就绪FD (list_head))
- 创建serverSocket (web服务都是基于tcp协议的, 在服务端都是一个serverSocket, 其在linux内部也会看作是一个文件, 其有自己的文件描述符fd, 记作ssfd)
- 调用epoll_ctl将ssfd添加到rb_root, 并为ssfd注册ep_poll_callback, 当fd就绪时, 将就绪fd记录到list_head链表里 (第一次就ssfd, 后面随着客户端连接, 可能会有越来越多的fd注册进来)
- 调用epoll_wait等待FD就绪, 判断list_head是否为空
 - 没有FD就绪, 重新执行epoll_wait
 - FD就绪
 - 是ssfd可读, accept接受客户端socket, 得到对应FD, 再执行epoll_ctl将其注册到监听fd红黑树中
 - 读取请求数据, 写出响应 (给客户端)

2.4、信号驱动IO

信号驱动IO是与内核建立SIGIO的信号关联并设置回调, 当内核有FD就绪时, 会发出SIGIO信号通知用户, 期间用户应用可以执行其他业务, 无需阻塞等待

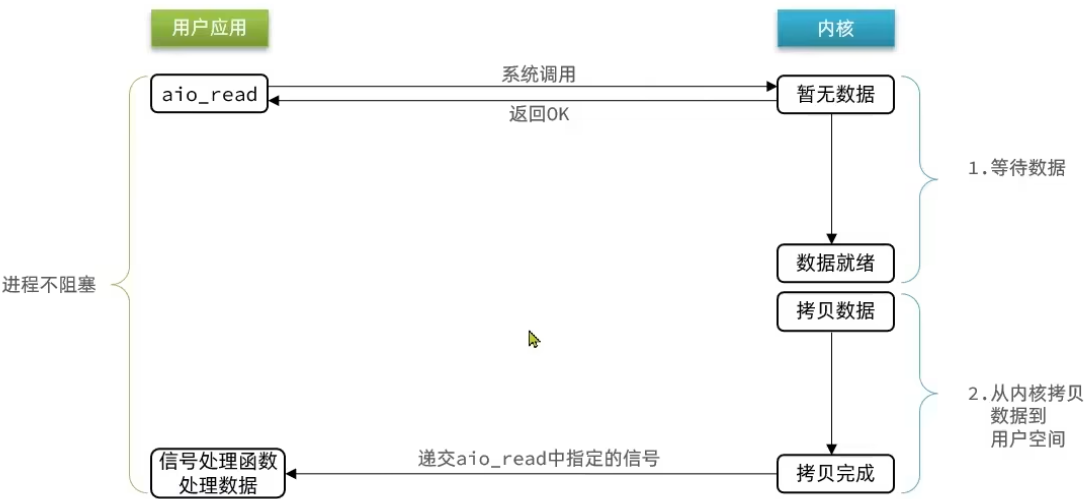
阻塞等待。



当有大量IO操作时，信号较多，SIGIO处理函数不能及时处理可能导致信号队列溢出，而且内核空间与用户空间的频繁信号交互性能也较低。

2.5、异步IO

异步IO的整个过程都是非阻塞的，用户进程调用完异步API后就可以去做其它事情，内核等待数据就绪并拷贝到用户空间之后才会递交信号，通知用户进程。



三、同步&异步

IO操作是同步还是异步，关键看数据在内核空间与用户空间的拷贝过程（数据读写的IO操作），也就是阶段二是同步还是异步：



图 6.6 五个 I/O 模型的比较